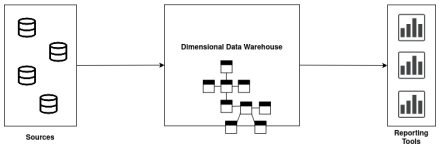
Data warehousing

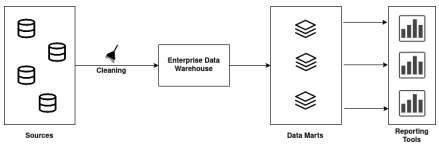
- Data warehouse is a repository of an organization's electronically stored data
- Data warehouses are designed to facilitate reporting and analysis
- A data warehouse stores a standardized, consistent, clean, and integrated form of data sourced from various operational systems in use in the organization
- Data sourced are structured in a way to specifically address the reporting and analytic requirements.

Data warehousing

- Bottom-up approach [Kimball 2002]:



- Top-down approach [Inmon 2002]:



Data warehousing
Kimball design

- Ralph Kimball design approach [Kimball 2002]:
  - The data model follows a bottom-up approach
  - Business process-oriented approach
    - First define data marts based on the business requirements
    - Build the data warehouse out of data marts
  - Each data mart uses a multi-dimensional modeling approach

Data warehousing
Kimball design advantages (I)

- R. Kimball design approach advantages [Kimball 2002]:
  - The dimensional modeling is fast to construct as no normalization is involved, which means swift execution of the initial phase of the data warehousing design process
  - A multi-dimensional modeling can easily be comprehended by data operators (users) because the denormalized structure also simplifies querying and analysis
  - Data warehouse focuses on individual business areas and processes rather than the whole enterprise

Data warehousing
Kimball design advantages (I)

- R. Kimball design approach advantages [Kimball 2002]:
  - It enables fast data retrieval from the data warehouse, as data is segregated into fact tables and dimensions.
  - A smaller team of designers and planners is sufficient for data warehouse management because data source systems are stable, and the data warehouse is process-oriented.
  - Query optimization is straightforward, predictable, and controllable.
  - Conformed dimensional structure for data quality framework, allowing business intelligence tools to deeper across several star schemas and generates reliable insights

Data warehousing
Kimball design disadvantages (I)

- R. Kimball design approach disadvantages [Kimball 2002]:
  - Data is not entirely integrated before reporting
  - Irregularities can occur when data is updated:
    - Denormalization adds redundant data
  - Performance issues may occur due to the addition of columns in the fact table, as these tables are quite in-depth.
    - The addition of new columns can expand the fact table dimensions, affecting its performance
    - Also, the dimensional data warehouse model becomes difficult to alter with any change in the business needs

Data warehousing
Kimball design disadvantages (II)

- R. Kimball design approach disadvantages [Kimball 2002]:
  - The model is business process-oriented
    - Does not focus on the enterprise as a whole
    - Cannot handle all the BI reporting requirements
  - The process of incorporating large amounts of legacy data into the data warehouse is complex

Data warehousing
Inmon design

- Bill Inmon modeling approach [Inmon 2002]:
  - The data model follows a top-down approach
    - Define the data warehouse
    - Afterwards define the data marts
  - A data warehouse is a: subject-oriented, integrated, nonvolatile, time-variant collection of data in support of management's decisions.
  - The Data Warehouse
    - Uses the normalized form for building entity structure, avoiding data redundancy as much as possible
    - Follows the normalized structure
    - Contains multiple data marts

Data warehousing
Inmon design advantages

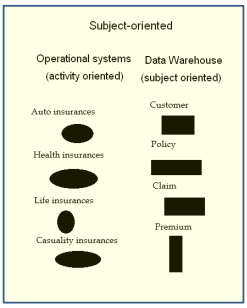
- B. Inmon design approach advantages [Inmon 2002]:
  - Data warehouse acts as a unified source of truth for the entire business, where all data is integrated
  - This approach has very low data redundancy.
    - Less irregular data updates
    - The process of adding new data to the data warehouse is more straightforward and less susceptible to failure
  - It simplifies business processes, as the logical model represents detailed business objects.
  - This approach offers greater flexibility
    - It's easier to update the data warehouse in case there's any change in the business requirements or source data
  - It can handle diverse enterprise-wide reporting requirements

Data warehousing
Inmon design disadvantages

- B. Inmon design approach disadvantages [Inmon 2002]:
  - Complexity increases as multiple tables are added to the data model with time
  - Skilled data operators (users) are required
    - Users need to understand the data warehouse data modeling
    - Expensive and challenging to find
  - The preliminary setup and delivery are time-consuming
  - Additional ETL process operation is required since data marts are created after the creation of the data warehouse
  - This approach requires experts to manage a data warehouse effectively

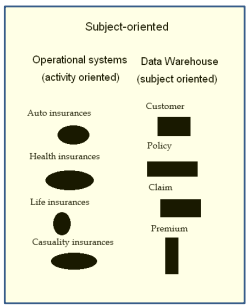
Data warehousing
Subject-Oriented

- Operational data systems of a company
  - Are organized considering the main activities
  - Are activity-oriented and not subject oriented
- A classical example in the literature is an insurance company where the ***main activities*** are:
  - Auto insurances
  - Health insurances
  - Life insurances
  - Casualty insurances



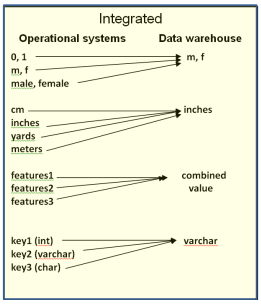
Data warehousing
Subject-Oriented

- For each activity there is possibly another software system managing data on the main ***subject areas***:
  - Policies
  - Customers
  - Claims
  - Premiums in the area
- There are possible separate databases, one for each activity, with similar but not identical structures
- When uploading data in the company data warehouse the data must first
  - Be ***restructured*** on these major subject areas
  - Integrates data on customers, policies, claims and premiums from each activity



Data warehousing
Integrated

- When preparing data for uploading in the data warehouse, one of the most important activities is the integration
- Data is loaded from operational sources and must be converted, summarized, re-keyed, etc., before loading it in the data warehouse
- Combine multiple encodings into a single one.
- For example
  - The gender may be encoded as (0, 1), (m, f), (male, female) in separate operational systems
  - If (m, f) is chosen as the data warehouse encoding, all data encoded using other convention must be converted



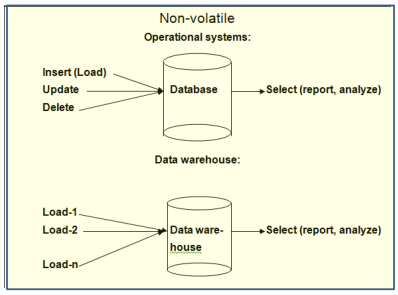
Data warehousing
Integrated

- Actions performed for data integration:
  - Chose a unique measure unit for each piece of information
    - For example, if length is measured in cm, inches, yards and meters in different operational systems, one unit must be chosen for the data warehouse and all other values must be converted
  - If the same object has in some data sources different values for the same attribute (e.g. description, name, features, etc.), these must be combined in a single one
  - If the same object has different keys in the source systems it must be re-keyed to have a single key in the data warehouse

Data warehousing
Non-volatile

- In usual operational systems data is updated or deleted to reflect the actual values
- In a data warehouse data is never updated and deleted:
  - After data is loaded, it stays there for future reporting
  - Similar to a snapshot it reflects the situation at a certain moment
- The next load operation
  - Data are added as new snapshots
  - The old snapshots are not changed
- The data warehouse is a sequence of snapshots that coexist
- In this way the data warehouse contains all the history of operational data (not only the operational data at a given moment)
- Because of this lack of change, once loaded, data in a data warehouse may be considered as read-only

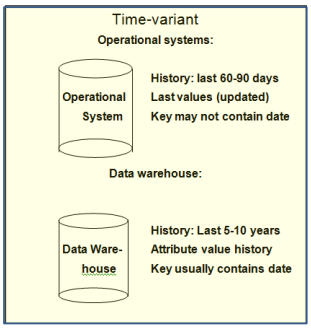
Data warehousing
Non-volatile



Data warehousing
Time variant

- As described above
  - A data warehouse contains a sequence of snapshots
  - Each snapshot being actual at a given moment of time
- Data Warehouse contains the whole history of a company thus it is possible to retrieve information from a time window (slice)
- Each unit of information is stamped or linked with the moment during which that information was accurate

Data warehousing



Data warehousing
Time variant

- In an operational system only the current data is kept
  - e.g., if a customer changes address, in the operational system old address is replaced (update) with the new one
- In the data warehouse all the successive information is kept
  - E.g., all successive addresses of a customer are kept
- The key (primary key) structure contains usually the date and sometimes the time
  - Date and time are very important for analyzing data and reporting

Data warehousing

- Requirements for a Data Warehouse [Kimball 2002]:
  - Information must be easy accessible
  - Information must be consistent
  - Flexibility
  - Security
  - Decision Support
  - User Acceptance

Data warehousing

- Why build a DW? [Kimball 2002]
  - "We have mountains of data in this company, but we can't access it."
  - "We need to slice and dice the data every which way."
  - "You've got to make it easy for business people to get at the data directly."
  - "Just show me what is important."
  - "It drives me crazy to have two people present the same business metrics at a meeting, but with different numbers."
  - "We want people to use information to support more fact-based decision making."

Data warehousing
Information must be easy accessible

- DW content must be understandable
- DW content must be intuitive or obvious to the non-database specialists, because they are the key users of the system
- Names must be meaningful (for data categories, features, attributes and so on, so the structure of the DW must be understandable for a non-specialist user)
- The DW must provide options for combining data, the process being known and referred to as *slicing and dicing*.
- The methods and tools for accessing data in the data warehouse must
  - Be simple
  - Be easy to use
  - Return the answers in a short time.

Data warehousing
Information must be consistent

- The process of fueling a data warehousing with data contains a step of preprocessing, where
  - Data is assembled from many sources, cleansed, quality assured
  - Data is released (published) to the users only when it is fit for usage
- An integration step is performed when data is load from operational sources to unify encodings, units of measure, keys, names and common values/features, etc.
- Common definitions for the contents of the data warehouse must be available for DW users

Data warehousing
Flexibility

- A data warehouse must be designed to be flexible considering the inevitable changes in computer science and engineering
- Its content must be structured in such a way that changes in the software and hardware platform must be possible
- Adding new data, reports, queries must be possible and must not interfere with existing ones

Data warehousing Security

- Because of its confidential content
  - The data warehouse must have the means for rejecting unauthorized access
  - Potential leaks of content may be harmful for the company if competitors have access to the data in the DW.

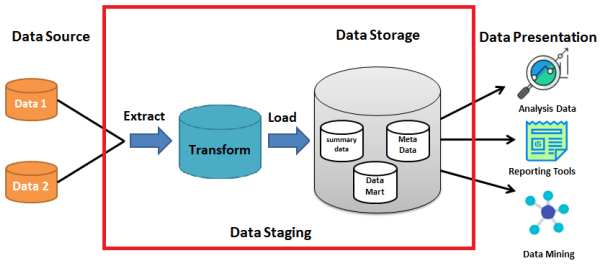
Data warehousing Decision support

- The primary goal of implementing a data warehouse in an organization is the decision support
- The ultimate output from a DW is the set of decisions based on its content, analyzed and presented in different ways to the decision makers
- The original label for a data warehouse and the tools around it was 'decision support system'

Data warehousing Acceptance

- The ultimate test for the success in implementing a data warehouse is the acceptance test
- If the business community does not continue to use it in the first six months after training, then the system has failed the acceptance test, no mater how bright is the technical solution
- It is possible to ignore using it because decisions may be adopted without a decision support system
- Key point in user acceptance is simplicity and user friendliness

DW architecture



DW architecture

- Operational Source Systems
  - These are the source of the data in the DW
  - Are placed outside of the data warehouse
- Data Staging Area
  - Here data is prepared (transformed) for loading in the data storage
  - This area is not accessible to the regular user
- Data Presentation
  - This part is what regular users see and consider to be a DW
  - Different Data Access Tools are available to the users
  - These tool used for analyzing and reporting
  - They provide the interface between the user and the DW

DW architecture

- The **data staging area** (DSA) of a data warehouse is :
  - Contains the two parts:
    - A data preprocessing pipeline (Extract-Transform-Load)
    - A storage area
  - A set of processes performing the so-called Extract-Transform-Load (ETL) operation:
    - Extract – Extracting data from Operational Source Systems
    - Transform – Integrating data from all sources
    - Load – Loading the data in the storage area and publishes the data to be accessed by the Data presentation area

DW architecture

- DSA contains everything between the operational source systems and the data presentation area
- This area is not accessible to the regular users of the data warehouse.
- Storing data in a DW (so also in DSA) may be done following two main approaches:
  1. The normalized approach
  2. The dimensional approach
- These approaches are not mutually exclusive, and there are other approaches.
- Dimensional approaches can involve normalizing data to a degree.

DW architecture Normalized approach

- In the ***normalized approach (extensively used in OLTP)***
  - Data are stored following database normalization rules
  - Tables are grouped by subject areas (data on customers, policies, claims and premiums for example)
- The main ***advantage*** of this approach is that loading data is straightforward because the philosophy of structuring data is the same for operational source systems and the data warehouse
- The main ***disadvantage*** of this approach is the number of joins needed to obtain meaningful information.
- A regular user needs also to have a good knowledge about the data in the DW and also a training period in obtaining de-normalized tables from normalized ones.
- Missing a join condition when performing a query may lead to Cartesian products instead of joins. In other words, regular user may need assistance from a database specialist to perform usual operations.

DW architecture
Dimensional approach

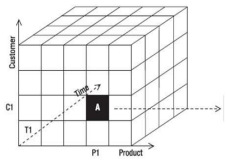
- Data is partitioned in two main categories:
  - Facts (numeric transaction data). In a retail example, the fact table contains quantity sold, total price, total cost, total gross profit
  - Dimensions (standardized contexts for facts). In a retail example, dimensions may be: product, date, time, location, customer, salesperson, etc.
  - Main approach OLAP
- ***Advantages*** of the dimensional approach are:
  - Data is easy to understand, easy to use, no need for assistance from a database specialist, speed in solving queries.
  - Data being de-normalized (or partially de-normalized) the number of joins needed for performing a query is lower than in the normalized approach
  - Joins between the fact table and its dimensions is easy to perform because the fact table contains surrogate keys for all involved dimension tables
- ***Disadvantages*** of dimensional approach:
  - The ETL process is harder to be performed because of the different philosophy in structuring data in the operational systems and the data warehouse: transform and load steps are more complicated than in the normalized approach
  - It is more difficult to modify the data warehouse scheme when the company changes its way to do business.

DW architecture
Data presentation area

- At the end of the ETL process prepared data is loaded in storage and published in the Data Presentation Area (DPA)
- After that moment, data is available for users for querying, reporting and other analytical applications
- Because regular users have access only to that area, they may consider the presentation area as being the data warehouse
- This area is structured as a series of integrated **data marts**, each presenting the data from a single business process

DW architecture
Data presentation area

- In the DPA data is presented and accessed in dimensional schemas
- We can imagine a hypercube with edges labeled with the dimensions, e.g., customer, product and time



DW architecture
Data access tools

- Almost all DW regular users (80% to 90%) will access the data via some prebuilt parameter-driven analytic applications.
- Generally, a user has four channels to interact with a DW:
  - Ah-hoc query tools
  - Report writers
  - Analytic applications
  - Modeling tools

DW architecture
Ah-hoc query tools

- By this channel the user obtains raw data verifying the conditions specified in the ad-hoc query
- For using this channel the user must have a good knowledge on the DW structure and on query language used
- This channel is for specialists and experienced users

DW architecture
Report writers

- This channel is at the same level as the first one
- Raw data is presented as a report
- Usually there are several pre-built reports that user may run without knowing the DW structure and query language
- Building new reports may need extra abilities

DW architecture
Analytic applications

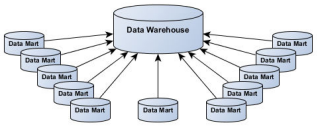
- Interactive reports
- Dashboards
- Scorecards
- Other reporting tools allowing users to access and analyze on data in the DW

DW architecture
Modeling tools

- In this category can be mentioned
  - Data mining products
  - Forecasting and scoring tools
  - Machine Learning tools
- At this level the result is not only a sophisticated report on existing data but also
  - Extracted new knowledge, patterns, and insights
  - Models for forecasting, classification, and clustering

DW architecture

- A ***data mart*** is defined as a repository of data gathered from operational data and other sources that is designed to serve a particular community of knowledge workers
- Data marts are analytical data stores designed to focus on specific business functions for a specific community within an organization.
- ***The data warehouse is created from the union of organizational data marts***.



OLAP

- OLAP (On-Line Analytical Processing) is a technology used to organize information in Data Warehouses which offers support for knowledge discovery useful in the decision-making process

OLAP

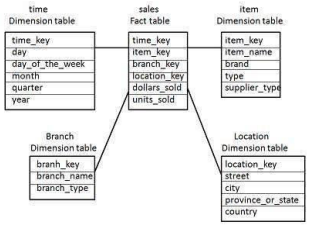
- Concepts:
  - **Hierarchy** is a logical tree-type structure used to organize the members of a dimension
  - **Level** is a data level of the hierarchy: the information from a hierarchy can be organized using different inferior or superior levels
  - **Member** is an element in a hierarchy
  - **Calculated Member** is a member whose value is computed during execution
  - **Dimension** is a set that contain one or more hierarchies
  - **Measure** (measurement or metric) is a set of values used when creating reports
  - **Cube** is a data structure that aggregates the measures by the levels and hierarchies of each analyzed dimension

OLAP

- In practice these concepts are used to create different schemas:
  - Star schema
  - Snowflake schema
  - Constellation schema

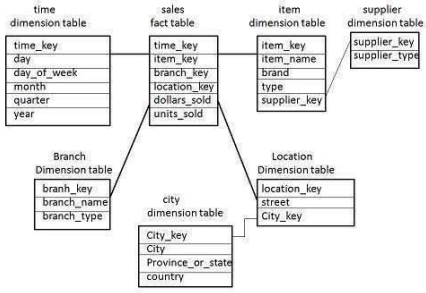
OLAP
Star Schema

- Each dimension is modeled as a table
- The dimension tables contain the set of attributes
- The Facts table constrains:
  - Foreign keys that reference the primary keys in the dimension tables
  - Measurements or metrics based on the dimension hierarchies



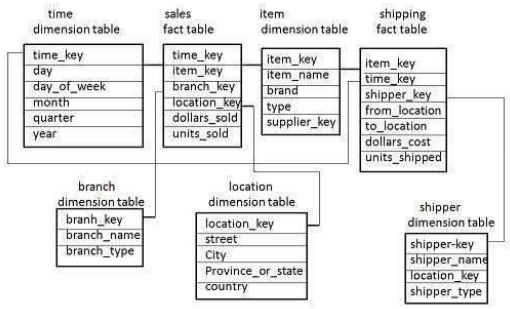
OLAP
Snowflake Schema

- The dimension are normalized
- The data from dimensions and hierarchies are split intro multiple tables



OLAP
Constellation Schema

- Contains multiple Fact Tables



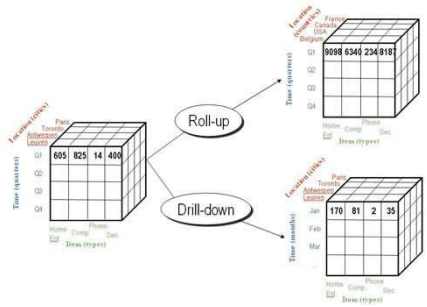
OLAP Operations

1. Roll-up is used to climb up a concept hierarchy for a dimension or by reducing dimensions
2. Drill-down is used o step down a concept hierarchy or by introducing a new dimension

- Roll-up is the reverse operation of Drill-down

OLAP Operations

- Roll-up and Drill-down



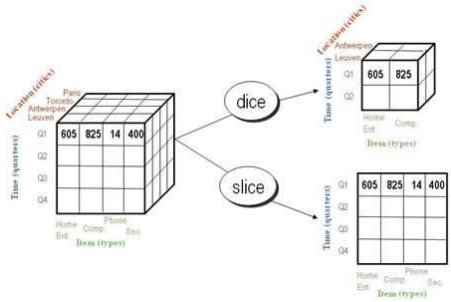
OLAP Operations

3. Slice operation selects one particular dimension from a given cube and provides a new sub-cube

4. Dice selects two or more dimensions from a given cube and provides a new sub-cube

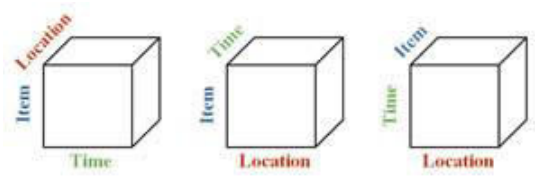
OLAP Operations

- Slice and Dice



OLAP Operations

5. Pivot (or rotate) is used to rotate the cube on one of the axes to visualize the date from different perspective

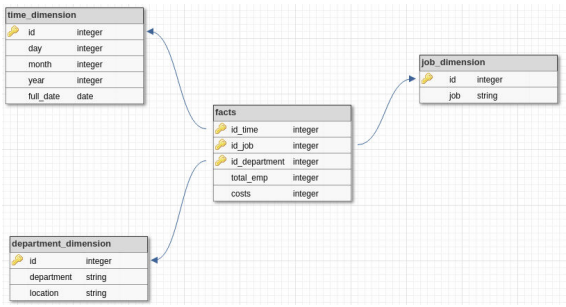


Example

- The following code is written for Oracle version >= 12c
- If you use an older version, when creating the tables you need to use sequences for autoincrementing the primary key
- The code in the slides is executable

Example

- Create multidimensional schema from the HR schema



Example

- Creating and populating the time_dimension

```
CREATE TABLE time_dimension(
  id INT generated as identity primary key,
  day INT,
  month INT,
  year INT,
  fulldate DATE
);

INSERT INTO time_dimension(fulldate, day, month, year)
SELECT
    DISTINCT hire_date,
    extract(day FROM hire_date),
    extract(month FROM hire_date),
    extract(year FROM hire_date)
FROM employees
ORDER BY hire_date;
```


Example

- Creating and populating the job_dimension:

```
CREATE TABLE job_dimension(
  id VARCHAR2(10) PRIMARY KEY,
  job varchar2(35)
);

INSERT INTO job_dimension(id, job)
SELECT DISTINCT job_id, job_title
FROM jobs
ORDER BY job_title;
```

Example

- Creating and populating the department_dimension

```
CREATE TABLE department_dimension(
   id INT generated AS identity PRIMARY KEY,
   department VARCHAR2(30),
   location VARCHAR2(30)
);

INSERT INTO department_dimension(department, location)
SELECT DISTINCT d.department_name, l.city
FROM departments d INNER JOIN locations l
     ON d.location_id = l.location_id
ORDER BY department_name;
```


Example

- Creating the facts table:
  - Add foreign keys to the primary keys from the dimension tables
  - Create a compound primary key from the foreign keys

```
CREATE TABLE facts(
  id_time INT,
  id_job VARCHAR2(10),
  id_departament INT,
  total_employees INT,
  costs float,
  CONSTRAINT fk_1 FOREIGN KEY(id_time) REFERENCES time_dimension(id),
  CONSTRAINT fk_2 FOREIGN KEY(id_job) REFERENCES job_dimension(id),
  CONSTRAINT fk_3 FOREIGN KEY(id_departament) REFERENCES department_dimension(id),
  CONSTRAINT pk_facts PRIMARY KEY(id_time, id_job, id_departament)
);
```


Example

- Populating the facts table (we need to get the keys from the dimension tables to keep the integrity)

```
INSERT INTO facts
SELECT
    td.id id_time,
    jd.id id_job,
    dd.id id_departament,
    count(employee_id) total_employees,
    sum(e.salary * (1 + NVL(e.commission_pct, 0))) costs
FROM employees e
  INNER JOIN departments d ON e.department_id = d.department_id
  INNER JOIN department_dimension dd ON dd.department = d.department_name
  INNER JOIN time_dimension td ON td.fulldate = e.hire_date
  INNER JOIN job_dimension jd ON jd.id = e.job_id
GROUP BY td.id, jd.id, dd.id;
```


Example

- Rollup to count the number of employees and the salary costs grouping by year, job and department

```
SELECT
  td.year,
  jd.job job,
  dd.department department,
  sum(total_employees) no_emp,
  sum(costs) sal_costs
FROM facts f
  INNER JOIN department_dimension dd ON f.id_departament = dd.id
  INNER JOIN time_dimension td ON f.id_time = td.id
  INNER JOIN job_dimension jd ON f.id_job = jd.id
GROUP BY
  rollup(td.year, jd.job, dd.department)
ORDER BY 1, 2, 3;
```


Example

- Drilldown to count the number of employees and the salary costs grouping by year, month, job and department

```
SELECT
  td.year,
  td.month,
  jd.job job,
  dd.department department,
  sum(total_employees) no_emp,
  sum(costs) sal_costs
FROM facts f
  INNER JOIN department_dimension dd ON f.id_departament = dd.id
  INNER JOIN time_dimension td ON f.id_time = td.id
  INNER JOIN job_dimension jd ON f.id_job = jd.id
GROUP BY
  rollup(td.year, td.month, jd.job, dd.department)
ORDER BY 1, 2, 3, 4;
```


Example

- Cube to count the number of employees and the salary costs grouping by year, month, job and department

```
SELECT
    td.year,
    td.month,
    jd.job job,
    dd.department department,
    sum(total_employees) no_emp,
    sum(costs) sal_costs
FROM facts f
    INNER JOIN department_dimension dd ON f.id_departament = dd.id
    INNER JOIN time_dimension td ON f.id_time = td.id
    INNER JOIN job_dimension jd ON f.id_job = jd.id
GROUP BY
    cube(td.year, td.month, jd.job, dd.department)
ORDER BY 1, 2, 3, 4;
```


Grouping sets

- If you want to group the data by multiple combinations of group by expressions, you may take one of two approaches:
  - The first approach is to create one grouped query per combination of expressions and merge the results using the UNION ALL operator.
  - The other approach is to use the GROUPING SETS operator along with the GROUP BY clause and define each grouping set within a single query.

Example

GROUPING SETS Equivalent of UNION ALL	
SELECT Col1, Col2, SUM(Col3) FROM Table1 GROUP BY GROUPING SETS ((Col1), (Col2))	SELECT Col1, NULL as Col2, SUM(Col3) FROM Table1 GROUP BY Col1 UNION ALL SELECT NULL as Col1, Col2, SUM(Col3) FROM Table1 GROUP BY Col2
GROUPING SETS ROLLUP Equivalents	
GROUP BY ROLLUP (Col1, Col2, Col3)	GROUP BY GROUPING SETS ((Col1, Col2, Col3) ,(Col1, Col2) ,(Col1) ,())
GROUPING SETS CUBE Equivalents	
GROUP BY CUBE (Col1, Col2, Col3)	GROUP BY GROUPING SETS ((Col1, Col2, Col3) ,(Col1, Col2) ,(Col1, Col3) ,(Col2, Col3) ,(Col1) ,(Col2) ,(Col3) ,())

Example

- Grouping Sets to count the number of employees and the salary costs grouping by the set (year, month) and (job, department)

```sql
SELECT
    td.year,
    td.month,
    jd.job job,
    dd.department department,
    sum(total_employees) no_emp,
    sum(costs) sal_costs
FROM facts f
    INNER JOIN department_dimension dd ON f.id_departament = dd.id
    INNER JOIN time_dimension td ON f.id_time = td.id
    INNER JOIN job_dimension jd ON f.id_job = jd.id
GROUP BY
    grouping sets( (td.year, td.month), (jd.job, dd.department));
```


Example

- Slice to get a facet of the cube (year = 2003)

```sql
SELECT
    td.year,
    td.month,
    dd.department department,
    sum(total_employees) no_emp,
    sum(costs) sal_costs
FROM facts f
    INNER JOIN department_dimension dd ON f.id_departament = dd.id
    INNER JOIN time_dimension td ON f.id_time = td.id
    INNER JOIN job_dimension jd ON f.id_job = jd.id
WHERE
    td.year = 2003
GROUP BY
    cube(td.year, td.month, jd.job, dd.department);
```


Example

- Dice to get a subcube (given by the condition in the where clause)

```sql
SELECT
    td.year,
    td.month,
    dd.department department,
    sum(total_employees) no_emp,
    sum(costs) sal_costs
FROM facts f
    INNER JOIN department_dimension dd ON f.id_departament = dd.id
    INNER JOIN time_dimension td ON f.id_time = td.id
    INNER JOIN job_dimension jd ON f.id_job = jd.id
WHERE
    td.year between 2002 AND 2003 AND
    td.month between 4 AND 6 AND
    lower(dd.department) IN ('accounting', 'shipping')
GROUP BY
    cube(td.year, td.month, dd.department);
```


Example

- Pivot change the column order in the projection, i.e., SELECT clause

```sql
SELECT
    dd.department department,
    td.year,
    td.month,
    sum(total_employees) no_emp,
    sum(costs) sal_costs
FROM facts f
    INNER JOIN department_dimension dd ON f.id_departament = dd.id
    INNER JOIN time_dimension td ON f.id_time = td.id
    INNER JOIN job_dimension jd ON f.id_job = jd.id
GROUP BY
    cube(td.year, td.month, jd.job, dd.department);
```